

project3/train.py

```
1  """
2  Training file for the models we implemented
3  """
4
5  from pathlib import Path
6
7  import torch
8  import torch.nn as nn
9  import torch.nn.utils
10 from torch.utils.data import DataLoader
11 from einops import rearrange
12 import wandb
13
14 from model import BigramLanguageModel, MiniGPT
15 from dataset import TinyStoriesDataset
16 from config import BigramConfig, MiniGPTConfig
17
18 def solver(model_name):
19     # Initialize the model
20     if model_name == "bigram":
21         config = BigramConfig
22         model = BigramLanguageModel(config)
23     elif model_name == "minigpt":
24         config = MiniGPTConfig
25         model = MiniGPT(config)
26     else:
27         raise ValueError("Invalid model name")
28
29     # Load the dataset
30     train_dataset = TinyStoriesDataset(
31         config.path_to_data,
32         mode="train",
33         context_length=config.context_length,
34     )
35     eval_dataset = TinyStoriesDataset(
36         config.path_to_data, mode="test", context_length=config.context_length
37     )
38
39     # Create the dataloaders
40     train_dataloader = DataLoader(
41         train_dataset, batch_size=config.batch_size, pin_memory=True
42     )
43     eval_dataloader = DataLoader(
44         eval_dataset, batch_size=config.batch_size, pin_memory=True
45     )
46
47     # Set the device
48     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
49
50     # Print number of parameters in the model
51     def count_parameters(model):
```

```

52         return sum(p.numel() for p in model.parameters() if p.requires_grad)
53     print("number of trainable parameters: %.2fM" % (count_parameters(model) /
1e6,))
54
55     # Initialize wandb if you want to use it
56     if config.to_log:
57         wandb.init(project="dl2_proj3")
58
59     # Create the save path if it does not exist
60     if not Path.exists(config.save_path):
61         Path.mkdir(config.save_path, parents=True, exist_ok=True)
62
63     """ """
64
65     You are required to implement the training loop for the model.
66
67     The code below is a skeleton for the training loop, for your reference.
68     You can fill in the missing parts or completely set it up from scratch.
69
70     Please keep the following in mind:
71     - You will need to define an appropriate loss function for the model.
72     - You will need to define an optimizer for the model.
73     - You are required to log the loss (either on wandb or any other logger you
prefer) every `config.log_interval` iterations.
74     - It is recommended that you save the model weights every
`config.save_iterations` iterations. You can also just save the model with the
best training loss.
75
76     NOTE :
77     - Please check the config file to see the different configurations you can set
for the model.
78     - The MiniGPT config has params that you do not need to use, these were added
to scale the model but are
79     not a required part of the assignment.
80     - Feel free to experiment with the parameters and I would be happy to talk to
you about them if interested.
81     """
82
83     """ """
84     # Define the loss function
85     loss_fn = nn.CrossEntropyLoss()
86
87     # Define the optimizer
88     optimizer = torch.optim.AdamW(model.parameters(), lr=config.learning_rate)
89     """ """
90
91     if config.scheduler:
92         scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
93             optimizer, T_0=2000, T_mult=2
94         )
95
96     model.train()
97     model.to(device)
98

```

```

99     for i, (context, target) in enumerate(train_dataloader):
100
101         train_loss = None # You can use this variable to store the training loss
for the current iteration
102         ### ===== TODO : START ===== ###
103         # Do the forward pass, compute the loss, do the backward pass, and update
the weights with the optimizer.
104         context = context.to(device, non_blocking=True)
105         target = target.to(device, non_blocking=True)
106
107         logits = model(context)
108         logits_flat = rearrange(logits, "b t v → (b t) v")
109         target_flat = rearrange(target, "b t → (b t)")
110
111         train_loss = loss_fn(logits_flat, target_flat)
112
113         optimizer.zero_grad()
114         train_loss.backward()
115         if config.to_clip_grad:
116             torch.nn.utils.clip_grad_norm_(model.parameters(),
config.gradient_clip)
117         optimizer.step()
118
119         train_loss = train_loss.item()
120
121         ### ===== TODO : END ===== ###
122
123         if config.scheduler:
124             scheduler.step()
125
126         del context, target # Clear memory
127
128         if i % config.log_interval == 0:
129
130             model.eval()
131             eval_loss = None # You can use this variable to store the evaluation
loss for the current iteration
132             ### ===== TODO : START ===== ###
133             # Compute the evaluation loss on the eval dataset.
134             # Hint:
135             # - Remember to manually break out of the evaluation loop as the
dataloader wraps around the dataset.
136             losses = []
137             num_eval_batches = 20
138             with torch.no_grad():
139                 for j, (ec, et) in enumerate(eval_dataloader):
140                     if j ≥ num_eval_batches:
141                         break
142                     ec = ec.to(device, non_blocking=True)
143                     et = et.to(device, non_blocking=True)
144                     el = model(ec)
145                     el_flat = rearrange(el, "b t v → (b t) v")
146                     et_flat = rearrange(et, "b t → (b t)")
147                     losses.append(loss_fn(el_flat, et_flat).item())

```

```
148     eval_loss = sum(losses) / len(losses) if losses else float("nan")
149
150     ### ===== TODO : END ===== ###
151
152     print(
153         f"Iteration {i}, Train Loss: {train_loss}",
154         f"Eval Loss: {eval_loss}",
155     )
156
157     if config.to_log:
158         wandb.log(
159             {
160                 "Train Loss": train_loss,
161                 "Eval Loss": eval_loss,
162             }
163         )
164
165     model.train()
166
167     # Save the model every config.save_iterations
168     if i % config.save_iterations == 0:
169         torch.save(
170             {
171                 "model_state_dict": model.state_dict(),
172                 "optimizer_state_dict": optimizer.state_dict(),
173                 "train_loss": train_loss,
174                 "eval_loss": eval_loss,
175                 "iteration": i,
176             },
177             config.save_path / f"mini_model_checkpoint_{i}.pt",
178         )
179
180     if i > config.max_iter:
181         break
```